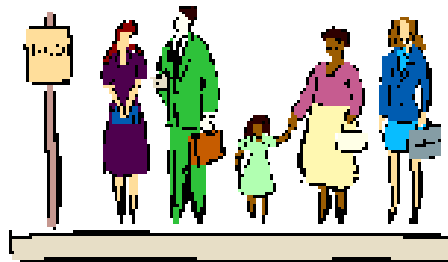


Data Structures



The Queue data structure

A queue is a data structure of *ordered* entries/items such that entries can only be inserted at one end (call the rear) and removed at the other end (call the front) – and the entry at the front of the queue is called the first entry

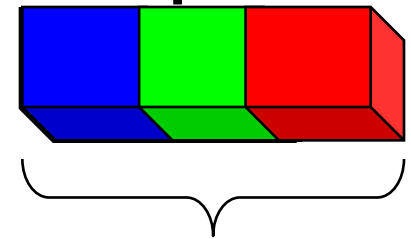


FIFO

A queue is a First-In/First-Out data structure. Entries are taken out of the queue in the same order that they were put into the queue

insert 
insert 
insert 
remove  (returned)
remove  (returned)
front  (returned)

The queue



length

FIFO: First-In-First-Out

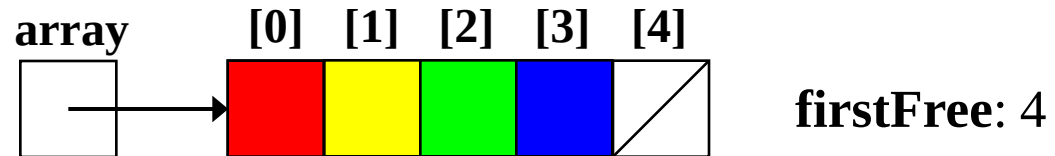
The Queue ADT

(informal specification)

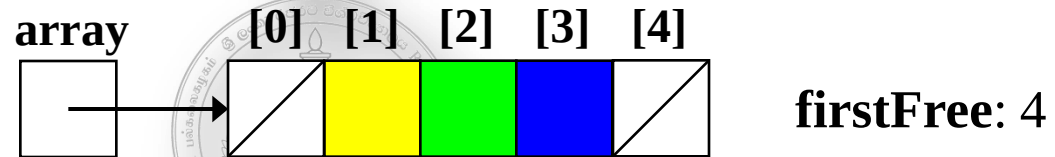
- ✖ An linear structure of fixed or variable size
- ✖ Elements are added to the *back*, and removed from the *front*
 - + **insert**: add an element to the back of the queue
 - + **remove**: remove an element from the front of the queue
 - + **getFront**: examine the element at the front of the queue (no removal)
- ✖ Other operations include:
 - + **isEmpty**: determines whether or not the queue is empty
 - + **length**: returns the number of element s in the queue
 - + **equals**: tests two queues for equality
 - + **toString**: converts a queue to a String
 - + **reverse**: turn-arround the queue contents

Inserting and removing: array based queue

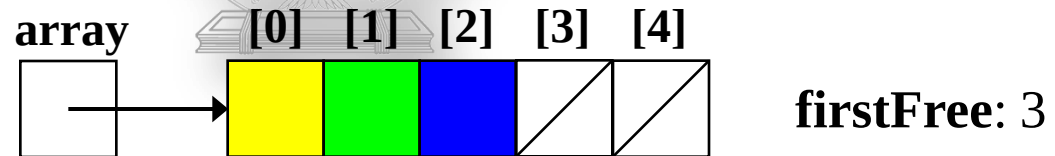
Inserting



Removing (1)



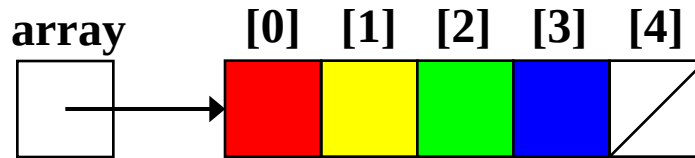
Removing (2)



Shuffling one place left: consider a queue with thousands of items!

Inserting and removing: a good strategy (circular array)

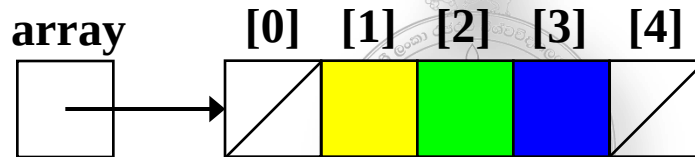
Inserting



first: 0

firstFree: 4

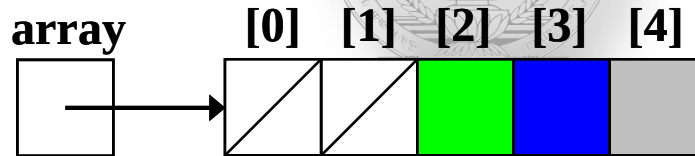
Removing



first: 1

firstFree: 4

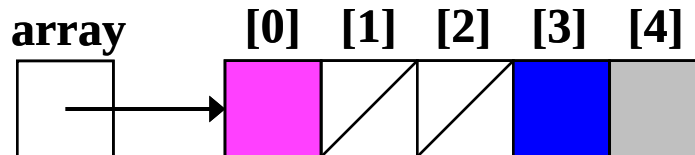
Ins. & Rem.



first: 2

firstFree: 0

Ins. & Rem.



first: 3

firstFree: 1

The method insert

- ✗ Need instance variables *first* and *firstFree*
- ✗ Also need instance variable *length*
 - ✚ Much easier than calculating the length from first and firstFree
- ✗ After adding the element:
 - ✚ Add one to firstFree, **in a circular way** two possible strategies:
 - ✚ $\text{firstFree}++$; if ($\text{firstFree} == \text{length}$) $\text{firstFree} = 0$;
 - ✚ $\text{firstFree} = (\text{firstFree} + 1) \% \text{length}$;
 - ✚ Increase length
- ✗ Don't forget to check whether the array is full

The method isEmpty

- The signature is **public *boolean* isEmpty()**
- Returns **true** if the queue is empty, or **false** if non-empty
- How can we implement this?



The method remove

- Removes the front of the queue
- The signature is **public object remove()**
 - Use the value of *first* to access and store the object in a temporary variable
 - Increment the value of *first* (in a circular way)
 - Decrement the value of *length*.
 - Return the value of the stored object.



The method getFront

- The signature is **public *Object* getFront()**
- It returns the front of the queue
- How can we implement this?



The method length

- The signature is **public *int* length()**
- It returns the number of elements in the queue
- How can we implement this?



The method Display

- The signature is **public void** display()
- Should return FRONT on one line, then each object in the array on separate lines
- Need to cope with wrapping around the end of the array

// PSEUDOCODE

If firstFree > first

 loop from first to firstFree - 1, printing elements

Else

 loop from first to end of list, printing elements

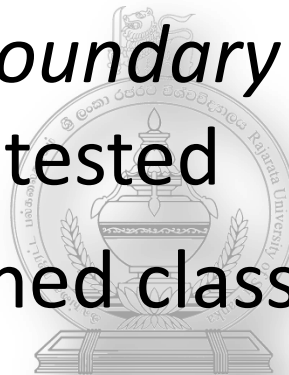
 loop from 0 to firstFree - 1, printing elements

End



Testing the queue so far

- Ensure that all methods and constructors are tested
- Ensure that the *boundary cases*, as well as the *typical cases*, are tested
- Use the user defined class as the object of the queue



The method equals:

When are two queues equal?

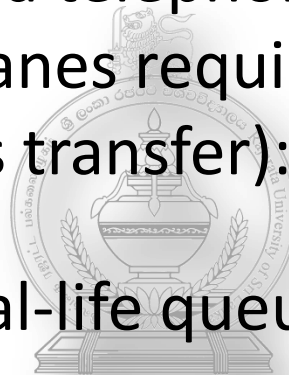
- ✗ The signature is **public boolean equals(Object* otherQueue)**
- ✗ Two queues are equal when their contents are equal (requires deep equality on each element of the array)
- ✗ Rather than examine the whole of both arrays, firstly check whether the lengths of the two queues are the same
- ✗ As before this is most easily done *outside* the Queue!
- ✗ Then iterate, similarly to the queue's toString
 - ✚ You'll need *two* location variables, because the first element in each queue might be in different locations

Use of the queue data structure: waiting for a service

- Application of Queues
- In a multitasking operating system, the CPU time is shared between multiple processes. At a given time, only one process is running, all the others are 'sleeping'. The CPU time is administered by the scheduler. The scheduler keeps all current processes in a queue with the active process at the front of the queue.

Use of the queue data structure: waiting for a service

- ✗ Jobs requiring a printer
- ✗ Client computers requiring a file or Web server
- ✗ Customers requiring a telephone operator
- ✗ Air traffic control: planes requiring clearance
- ✗ EFT (electronic funds transfer): transactions requiring handling
- ✗ Simulation of any real-life queuing situation
 - + Post office
 - + Supermarket
 - + Clinic
 - + ...



Priority Queue

- FIFO rules of a queue are relaxed
- Customers or jobs with higher priority are pushed to front of queue
- A priority queue is *least-first-out*
 - The “smallest” element is the first one removed
 - The definition of “smallest” is up to the programmer

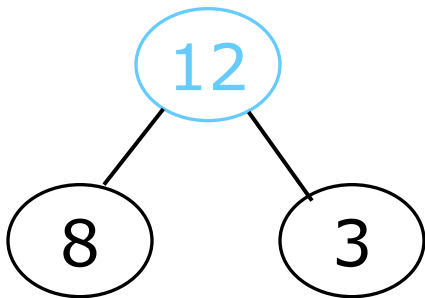
Priority Queue

- If there are several “smallest” elements, the implementer must decide which to remove first
 - Remove any “smallest” element (don’t care which)
 - Remove the first one added
- To implement:
 - use an ordinary linked list, which keeps the items in order from the highest to lowest priority
 - use a treelike structure

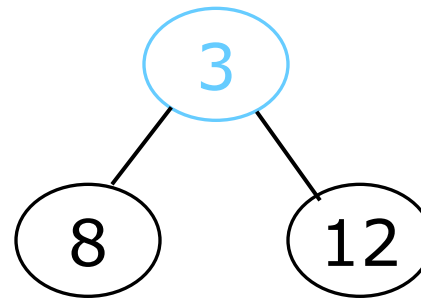


Heap implementation

- A priority queue can be implemented as a heap
- In order to do this, we have to define the *heap property*
 - In Heapsort, a node has the heap property if it is ***at least as large as its children***
 - For a priority queue, we will define a node to have the heap property if it is at least as *small* as its children (since we are using smaller numbers to represent higher priorities)

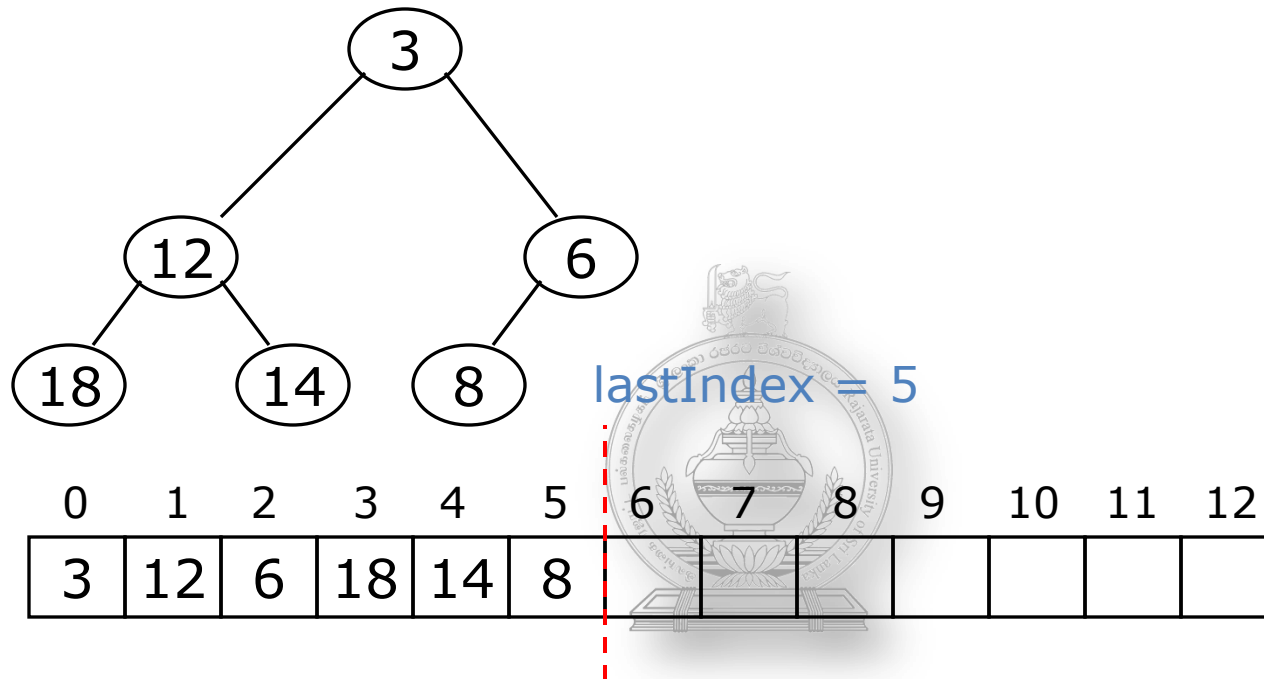


Heapsort: Blue node (12)
has the heap property



Priority queue: Blue node
(3) has the heap property

Array representation of a heap



- Left child of node i is $2*i + 1$, right child is $2*i + 2$
 - Unless the computation yields a value larger than lastIndex , in which case there is no such child
- Parent of node i is $(i - 1)/2$
 - Unless $i == 0$

Using the heap

- To add an element:
 - Increase `lastIndex` and put the new value there
 - Reheap the newly added node
 - This is called up-heap bubbling or percolating up
 - Up-heap bubbling requires $O(\log n)$ time
- To remove an element:
 - Remove the element at location `0`
 - Move the element at location `lastIndex` to location `0`, and decrement `lastIndex`
 - Reheap the new root node (the one now at location `0`)
 - This is called down-heap bubbling or percolating down
 - Down-heap bubbling requires $O(\log n)$ time
- Thus, it requires $O(\log n)$ time to add and remove an element

